The Journal of Open Source Software

ChemInformant: A Robust and Workflow-Centric Python Client for High-Throughput PubChem Access

Zhiang He[1]

1 Independent Researcher

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Evan Spotte-Smith

Reviewers:

- @cthoyt
- @Sulstice
- @LIVazquezS

Submitted: 14 April 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

ChemInformant is a Python client engineered for programmatic access to PubChem, specifically targeting high-throughput and automated data retrieval tasks. Its architecture streamlines the entire workflow from data acquisition to analysis by directly converting large, mixed-type lists of chemical identifiers into analysis-ready Pandas DataFrames (The pandas development team, 2025). To ensure operational resilience, the package natively integrates a suite of robustness features, including persistent HTTP caching, automatic rate-limiting with exponential backoff retries, and runtime data validation using Pydantic (Colvin & others, 2025). By systematically addressing critical limitations in existing tools—such as network instability and inefficient batch processing—and offering up to a 48-fold performance increase, ChemInformant delivers a significantly more reliable and efficient component for the modern Python cheminformatics ecosystem.

Statement of Need

As these workflows become increasingly automated and scaled, researchers encounter recurring challenges with existing client libraries, primarily concerning network reliability, batch processing capabilities, and a lack of workflow-centric API design.

First, network stability is a significant operational concern. The PubChem API service (Kim et al., 2018) enforces dynamic rate limits (e.g., 5 requests per second) and may return HTTP 503 (Server Busy) errors during periods of high traffic (National Center for Biotechnology Information, 2024). Many existing clients, such as PubChemPy (Swain, 2017), do not include built-in mechanisms for automatic request throttling or exponential backoff retries. This can lead to script fragility in automated environments, often requiring users to implement manual delays. Furthermore, the general absence of a persistent caching layer results in redundant network requests for repeated queries, which increases latency and unnecessarily consumes API usage quotas.

Second, limitations in handling heterogeneous inputs and providing clear error feedback for batch operations create inefficiencies in high-throughput data processing. Scientific workflows often involve large lists of mixed-type identifiers (e.g., a combination of names, CIDs, and SMILES). Typically, existing tools require users to pre-process these lists into homogeneous groups, adding a preparatory step to the workflow. Additionally, their fault tolerance for batch queries can be limited; a single invalid identifier may cause an entire operation to fail or return incomplete data without explicitly indicating which inputs were problematic. The lack of structured partial success and failure reporting complicates error diagnostics and can affect the reliability of data acquisition pipelines.

Furthermore, the architecture of existing client libraries underscores the need for a shift toward more modern, workflow-centric designs. While tools such as PubChemPy (Swain, 2017) are cornerstones in the field, they were designed in an era that prioritized the direct implementation of core API functionalities. Consequently, features like automatic retries for network errors,

persistent caching with sensible cross-platform defaults, and fine-grained error handling were often left for the developer to implement. This paradigm requires users building automated workflows to write significant boilerplate code to manage concerns such as API rate-limiting and cache path configuration, thereby diverting focus from their core scientific objectives.

These design decisions directly compound the aforementioned challenges in network stability and batch processing efficiency. In the absence of built-in fault-tolerance mechanisms, processing large, heterogeneous datasets becomes precarious, as a single invalid input can be sufficient to halt an entire workflow. ChemInformant was developed specifically to address these gaps. By natively integrating robustness, efficiency, and a "zero-configuration-first" philosophy at the architectural level, it provides a more resilient and streamlined modern tool, enabling researchers to concentrate on scientific analysis rather than the low-level mechanics of data acquisition.

State of the Field and Comparison

To contextualize `ChemInformant`, a comparative analysis was conducted against related tools, including PubChemPy, PubChemR (Korkmaz et al., 2025), webchem (Szöcs et al., 2020), ChemSpiPy (Swain, 2018), and PubChem4J (Southern & Griffin, 2011). **Table 1** outlines the features of these tools across several dimensions relevant to automated research workflows. The maintenance status of several key libraries is particularly noteworthy: PubChemPy has not had a formal release since 2017, and ChemSpiPy has been inactive since 2018, which underscores the need for a modern, actively maintained tool.

Table 1: Comparative analysis of key features in mainstream chemical information clients.

Key Feature	**ChemInformant**	PubChemPy	PubChemR	webchem	ChemSpiPy	PubChem4J
Platform	**Python**	Python	R	R	Python	Java
Primary Database	**PubChem**	PubChem	PubChem	Multi-DB	ChemSpider	PubChem
Persistent Caching[1]	**Yes**	No	No	No	No	N/A
Rate-Limiting & Retries[2]	**Yes**	No	No	Partial	No	N/A
Batch Retrieval	**Yes**	Partial	Partial	Partial	Partial	Yes
Mixed Identifier Support	**Yes**	No	No	No	No	N/A
Fault Tolerance[3]	**Yes**	No	No	No	No	N/A
Automatic Pagination	**Yes**	Partial	No	No	No	N/A
Runtime Type Safety	**Yes**	No	Partial	No	No	Yes
Project Activity	**Active**	Inactive	Active	Active	Inactive	Archived

Notes: [1] **Persistent Caching**: Improves speed on repeated queries by storing results locally. [2] **Rate-Limiting & Retries**: Automatically manages API request limits and server errors, enhancing automation robustness. [3] **Fault Tolerance**: Provides structured status reporting for each item in a batch query, avoiding complete failure on a single error.

DRAFT

Performance Evaluation

To quantify the performance of `ChemInformant`'s design, a benchmark test was performed to retrieve six different properties for a list of 285 drug names. For a direct, same-platform comparison, `PubChemPy` was selected as the baseline. Because `PubChemPy`'s batch property interface does not accept names as input, the test procedure first resolved all names to CIDs. The performance of both libraries was then timed on processing the resulting list of CIDs.

Table 2 summarizes the performance data for the 280 successfully resolved compounds.

Table 2: Performance benchmark results.

Scenario	Time (s)	Speed-up (vs. PubChemPy)
PubChemPy (batch interface)	6.50	1× (Baseline)
ChemInformant — Cold Cache	**1.40**	**4.6×**
ChemInformant — Warm Cache	**0.135**	**48×**

The initial `ChemInformant` batch query completed in **1.4 seconds**, a **4.6-fold** increase in speed compared to `PubChemPy`'s 6.5 seconds. A subsequent query from the cache finished in **135 milliseconds**, representing an overall 48-fold speed-up relative to the baseline. This sub-200ms response time is suitable for interactive applications. The script used for this benchmark is available in the project repository.

Example Usage

`ChemInformant` offers a layered API, with convenience functions for single lookups and a core engine for batch processing.

Convenience API Example (for single lookups):

```python
import ChemInformant as ci

# Retrieve a single property for one identifier
cas_number = ci.get_cas("ibuprofen")
# > '15687-27-1'
```

Core API Example (for batch data analysis):

```python
# Retrieve multiple properties for a list of mixed-identifier types
df = ci.get_properties(
    identifiers=["aspirin", "caffeine", 1983], # Mix of names and a CID
    properties=["molecular_weight", "xlogp", "cas"]
)
# The returned DataFrame is formatted for direct use in downstream analysis tools
print(df)
```

A more detailed user manual, including examples of how to integrate with other analysis tools, is available in the project repository.

Acknowledgements

The author thanks PubChem for providing open data services (Kim et al., 2023). The author also acknowledges the developers of the open-source libraries upon which ChemInformant is built, including requests, pandas (The pandas development team, 2025), pydantic (Colvin & others, 2025), requests-cache (Haritonov & Cook, 2024), and pystow (Hoyt et al., 2025).

DRAFT

References

Colvin, S., & others. (2025). *Pydantic* (Version 2.11.7). Zenodo. https://doi.org/10.5281/zenodo.15662245

Haritonov, R., & Cook, J. W. (2024). *Requests-cache: Persistent HTTP cache for python `requests`* (Version 1.2.1). GitHub. https://github.com/requests-cache/requests-cache

Hoyt, C. T., Obraczka, D., Berrendorf, M., & Baird, S. G. (2025). *Cthoyt/pystow: v0.7.1* (Version v0.7.1). Zenodo. https://doi.org/10.5281/zenodo.15720682

Kim, S., Chen, J., Cheng, T., Gindulyte, A., He, J., He, S., Li, Q., Shoemaker, B. A., Thiessen, P. A., Yu, B., Zaslavsky, L., Zhang, J., & Bolton, E. E. (2023). PubChem 2023 update. *Nucleic Acids Research*, *51*(D1), D1373–D1380. https://doi.org/10.1093/nar/gkac956

Kim, S., Thiessen, P. A., Cheng, T., Yu, B., & Bolton, E. E. (2018). An update on PUG-REST: RESTful interface for programmatic access to PubChem. *Nucleic Acids Research*, *46*(W1), W563–W570. https://doi.org/10.1093/nar/gky294

Korkmaz, S., Yamasan, B. E., & Goksuluk, D. (2025). *PubChemR: An r client for the PubChem API* (Version 2.1.4). Comprehensive R Archive Network (CRAN). https://doi.org/10.32614/CRAN.package.PubChemR

National Center for Biotechnology Information. (2024). *PubChem programmatic access usage policy*. https://pubchem.ncbi.nlm.nih.gov/docs/programmatic-access

Southern, M. R., & Griffin, P. R. (2011). A java API for working with PubChem datasets. *Bioinformatics*, *27*(5), 741–742. https://doi.org/10.1093/bioinformatics/btq715

Swain, M. (2017). *PubChemPy v1.0.4* (Version 1.0.4). Zenodo. https://doi.org/10.5281/zenodo.541438

Swain, M. (2018). *ChemSpiPy: A python wrapper for the ChemSpider API* (Version 2.0.0). https://pypi.org/project/ChemSpiPy/2.0.0

Szöcs, E., Stirling, T., Scott, E. R., Scharmüller, A., & Schäfer, R. B. (2020). Webchem: An r package to retrieve chemical information from the web. *Journal of Statistical Software*, *93*(13), 1–17. https://doi.org/10.18637/jss.v093.i13

The pandas development team. (2025). *Pandas-dev/pandas: pandas* (Version 2.3.1). Zenodo. https://doi.org/10.5281/zenodo.15831829

DRAFT